Name: Sanjog Poojary

Roll no : 38

Experiment 8

Aim

Experiment to implement Association mining algorithm(Apriori) using Rapid Miner and Python.

Theory

- Find frequent itemsets using a minimum support value.
- Generate association rules from the frequent itemsets using a minimum confidence value. 3.Important Terms
- Itemset: A group of one or more items.
- Frequent Itemset: A combination of items that appears frequently and satisfies the minimum support threshold.
- Association Rule: A rule written as A ⇒ B, which shows that when A occurs, B may also occur.
- Support: The percentage or proportion of transactions that contain a particular item or itemset.
- Confidence: The probability that B occurs in a transaction when A is already present.
- Lift: A measure used to check how strongly two itemsets are associated compared with what would be expected if they were independent.
- For mulas Support: Support(A) = Number of transactions containing A / Total number of transactions It tells us how frequently an item or itemset appears in the dataset. Confidence: Confidence(A ⇒ B) = Support(A ∪ B) / Support(A) It tells us how often B appears when A is present.

Lift: Lift(A ⇒ B) = Confidence(A ⇒ B) / Support(B)

- Lift greater than 1: A and B have a positive association.
- Lift equal to 1: A and B are independent.
- Lift less than 1: A and B have a negative association. 5.Dataset Description The Groceries dataset is obtained from Kaggle and contains grocery purchase records. Each record represents an item purchased by a customer on a particular date. The dataset can be used to study customer purchasing behaviour and discover products that are frequently bought together. The main columns are:
- Member_number: Identifies the customer or member who made the purchase.
- Date: Specifies the date of the purchase.
- itemDescription: Contains the name of the purchased grocery item, such as whole milk or tropical fruit. The dataset does not contain product prices, quantities, or detailed customer information. According to the reference description, it contains 14,963 unique transactions and 167 unique items. 6.Use of the Apriori Algorithm To apply Apriori to the grocery dataset, transactions are grouped using the member number and date. A transaction matrix is then created, where rows represent transactions and columns represent grocery items. A value of 1 indicates that an item is present in a transaction, while 0 indicates that it is absent. The Apriori algorithm is applied to identify frequent itemsets and generate association rules. These rules help discover purchasing patterns, such as when

customers who buy one product also tend to buy another product. This dataset contains transactional data from a grocery store, capturing customer purchases of various grocery items. The data is likely collected from a retail environment (specific details about the store or location are not provided in the source). Each record represents a single item purchased as part of a customer's transaction, identified by a member number and date. The dataset is suitable for market basket analysis and association mining, such as applying the Apriori algorithm to discover patterns in customer buying behavior.

Dataset Overview

- **Dataset:** JioHotstar Preprocessed Dataset (33,171 records and 24 columns).
- **Transaction items:** seven binary behaviour items derived from the viewing features: High_Watch, Strong_Hook, High_Pacing, High_Visual_Intensity, Skipped_Intro, High_Cognitive_Load and Night_Watch_Safe.
- **Thresholds:** avg_watch_percentage ≥ 70; hook_strength ≥ 7; pacing_score ≥ 7; visual_intensity ≥ 7; skip_intro = 1; cognitive_load ≥ 7; night_watch_safe = 1.

Implementation (Python)

Output

The JioHotstar dataset was selected and uploaded for analysis. This dataset contains episode characteristics and viewer-behaviour information that will be used to discover frequent patterns and associations.

```
from google.colab import files

uploaded = files.upload()
```

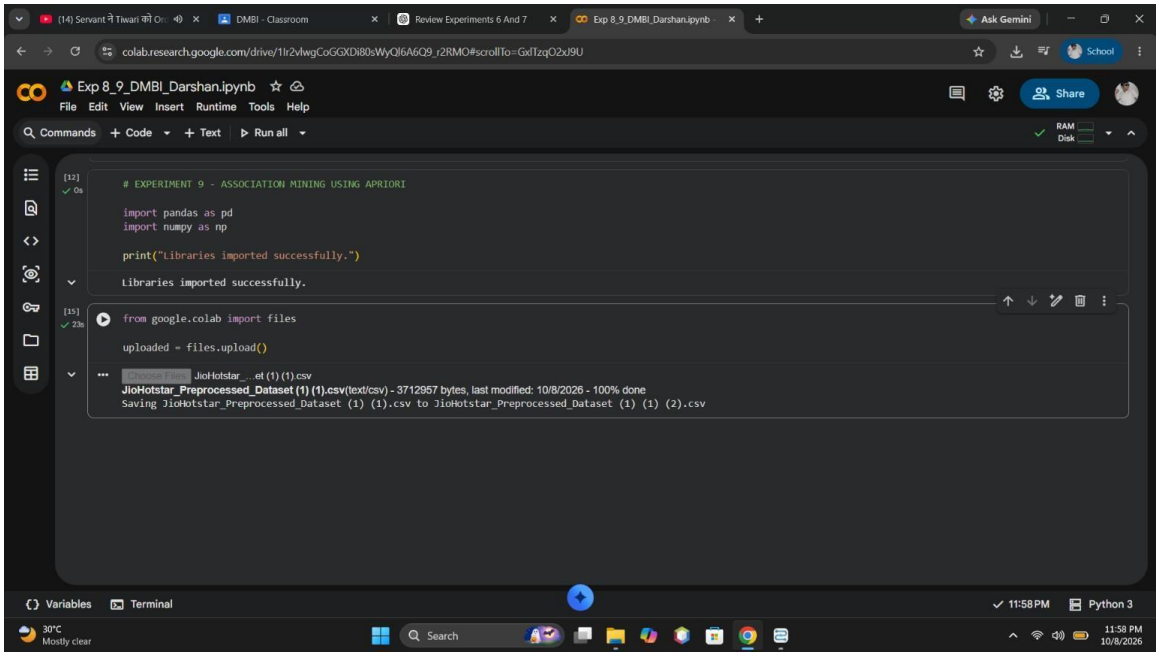


Figure 1: JioHotstar dataset uploaded in Google Colab

```
# EXPERIMENT 8 - ASSOCIATION MINING USING APRIORI
```

```
import pandas as pd
import numpy as np

print("Libraries imported successfully.")
```

```
# Load JioHotstar dataset

file_name = "JioHotstar_Preprocessed_Dataset (1) (1) (2).csv"

df = pd.read_csv(file_name)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())
```



Figure 2: Dataset loaded into Python: shape (33171, 24) and column names

The dataset was loaded into Python for processing. Loading the data makes it available for inspection, transformation, and association rule mining.

```
# Check data types and missing values

print("Data types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())
```

```
# Check data types and missing values

print("Data types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())
```

```
Data types:
show_id                    int64
title                     object
platform                  object
genre                     object
release_year             float64
season_number              int64
episode_number             int64
episode_duration_min       int64
pacing_score               int64
hook_strength              int64
dialogue_density          object
visual_intensity           int64
avg_watch_percentage       int64
pause_count                int64
rewind_count               int64
skip_intro                 int64
cognitive_load             int64
attention_required        object
night_watch_safe           int64
drop_off                   int64
drop_off_probability     float64
retention_risk            object
dataset_version           object
watch_engagement_level    object
dtype: object

Missing values:
show_id                   0
title                     0
platform                  0
genre                     0
release_year              0
season_number             0
episode_number            0
episode_duration_min      0
pacing_score              0
hook_strength             0
```

Figure 3: Data types and missing-value check

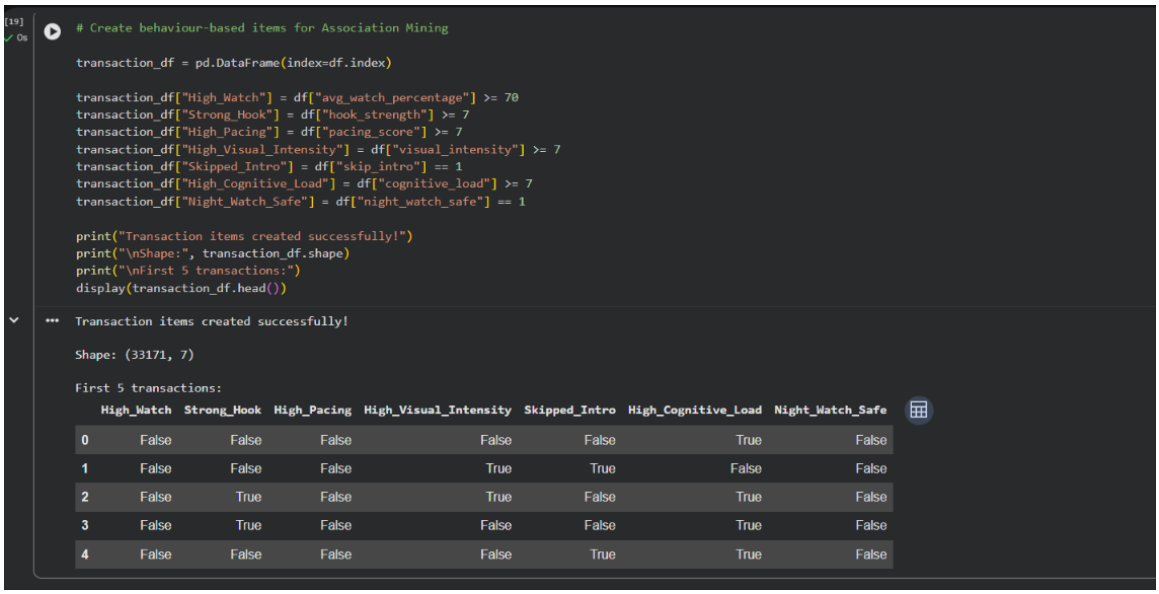
The dataset was inspected to understand its structure, column names, data types, and sample records. This step helps verify that the data is loaded correctly and identify relevant attributes for further analysis.

```
# Create behaviour-based items for Association Mining

transaction_df = pd.DataFrame(index=df.index)

transaction_df["High_Watch"] = df["avg_watch_percentage"] >= 70
transaction_df["Strong_Hook"] = df["hook_strength"] >= 7
transaction_df["High_Pacing"] = df["pacing_score"] >= 7
transaction_df["High_Visual_Intensity"] = df["visual_intensity"] >= 7
transaction_df["Skipped_Intro"] = df["skip_intro"] == 1
transaction_df["High_Cognitive_Load"] = df["cognitive_load"] >= 7
transaction_df["Night_Watch_Safe"] = df["night_watch_safe"] == 1

print("Transaction items created successfully!")
print("\nShape:", transaction_df.shape)
print("\nFirst 5 transactions:")
display(transaction_df.head())
```



```
# Create behaviour-based items for Association Mining

transaction_df = pd.DataFrame(index=df.index)

transaction_df["High_Watch"] = df["avg_watch_percentage"] >= 70
transaction_df["Strong_Hook"] = df["hook_strength"] >= 7
transaction_df["High_Pacing"] = df["pacing_score"] >= 7
transaction_df["High_Visual_Intensity"] = df["visual_intensity"] >= 7
transaction_df["Skipped_Intro"] = df["skip_intro"] == 1
transaction_df["High_Cognitive_Load"] = df["cognitive_load"] >= 7
transaction_df["Night_Watch_Safe"] = df["night_watch_safe"] == 1

print("Transaction items created successfully!")
print("\nShape:", transaction_df.shape)
print("\nFirst 5 transactions:")
display(transaction_df.head())
```

```
Transaction items created successfully!

Shape: (33171, 7)

First 5 transactions:
```

	High_Watch	Strong_Hook	High_Pacing	High_Visual_Intensity	Skipped_Intro	High_Cognitive_Load	Night_Watch_Safe
0	False	False	False	False	False	True	False
1	False	False	False	True	True	False	False
2	False	True	False	True	False	True	False
3	False	True	False	False	False	True	False
4	False	False	False	False	True	True	False

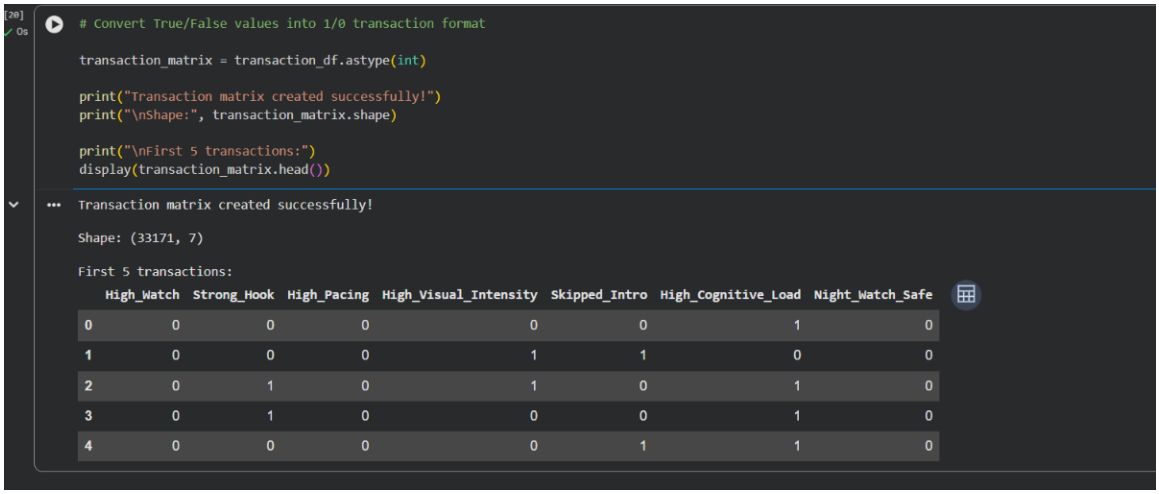
Figure 4: Behaviour-based transaction items created (True/False)

Relevant viewing behaviours were converted into transaction items, such as high watch percentage, strong hook, high pacing, high visual intensity, intro skipping, high cognitive load, and night-watch safety. These items represent the presence of specific characteristics in a record.

```
# Convert True/False values into 1/0 transaction format

transaction_matrix = transaction_df.astype(int)

print("Transaction matrix created successfully!")
print("\nShape:", transaction_matrix.shape)
print("\nFirst 5 transactions:")
display(transaction_matrix.head())
```



```
# Convert True/False values into 1/0 transaction format

transaction_matrix = transaction_df.astype(int)

print("Transaction matrix created successfully!")
print("\nShape:", transaction_matrix.shape)

print("\nFirst 5 transactions:")
display(transaction_matrix.head())
```

```
Transaction matrix created successfully!

Shape: (33171, 7)

First 5 transactions:
```

	High_Watch	Strong_Hook	High_Pacing	High_Visual_Intensity	Skipped_Intro	High_Cognitive_Load	Night_Watch_Safe
0	0	0	0	0	0	1	0
1	0	0	0	1	1	0	0
2	0	1	0	1	0	1	0
3	0	1	0	0	0	1	0
4	0	0	0	0	1	1	0

Figure 5: Transaction matrix with 1/0 values

A transaction matrix was prepared in which rows represent records and columns represent behavioural items. Binary values indicate whether an item is present or absent. This matrix provides the required input format for frequent itemset mining.

```
# Check the frequency of each behaviour item

item_counts = transaction_matrix.sum().sort_values(ascending=False)

item_percentage = (item_counts / len(transaction_matrix) * 100).round(2)

frequency_table = pd.DataFrame({
    "Item_Count": item_counts,
    "Support_Percentage": item_percentage
})

print("Item frequency:")
display(frequency_table)
```

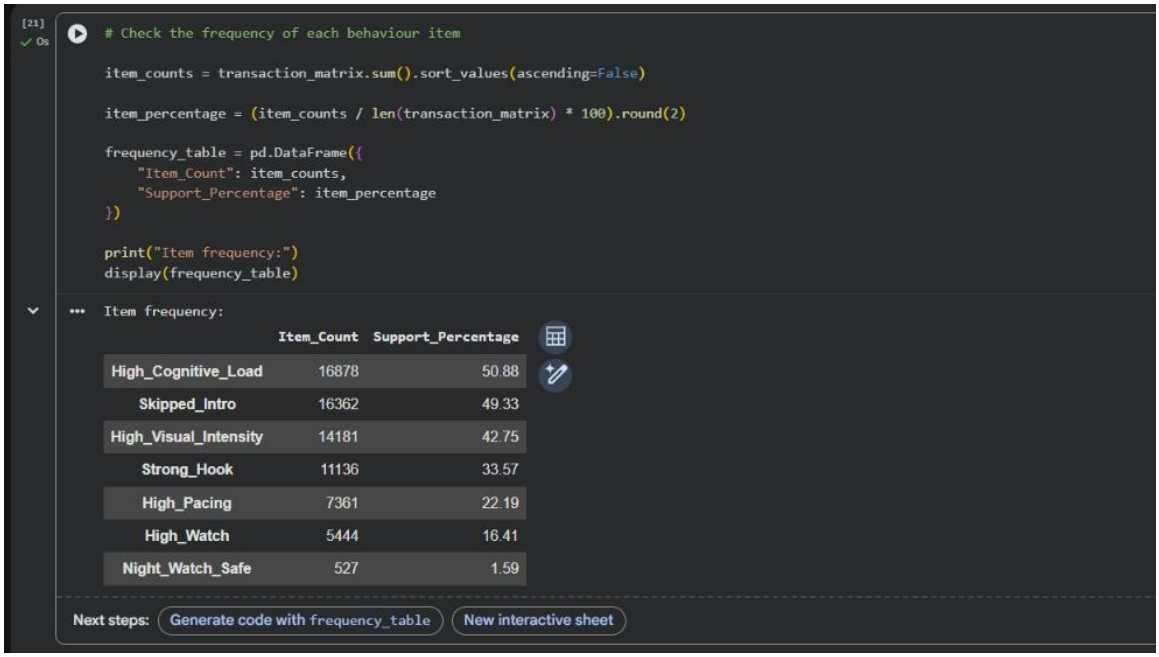


Figure 6: Item count and support percentage of each behaviour item

The frequency and support of individual behavioural items were calculated. Support represents the proportion of records containing an item. This analysis helps identify common viewing behaviours in the dataset.

```
# Install mlxtend for Apriori and association rules

!pip install mlxtend -q

print("mlxtend installed successfully!")
```

The Apriori algorithm and its required Python library functions were imported or prepared for execution. Apriori identifies frequent itemsets by checking whether their support meets a specified minimum threshold.

```
from mlxtend.frequent_patterns import apriori, association_rules

# Find frequent itemsets using Apriori
frequent_itemsets = apriori(
    transaction_matrix,
    min_support=0.05,
```

```
    use_colnames=True
)

print("Frequent itemsets generated successfully!")
print("Number of Frequent Itemsets:", len(frequent_itemsets))

display(frequent_itemsets.sort_values(
    by="support",
    ascending=False
))
```

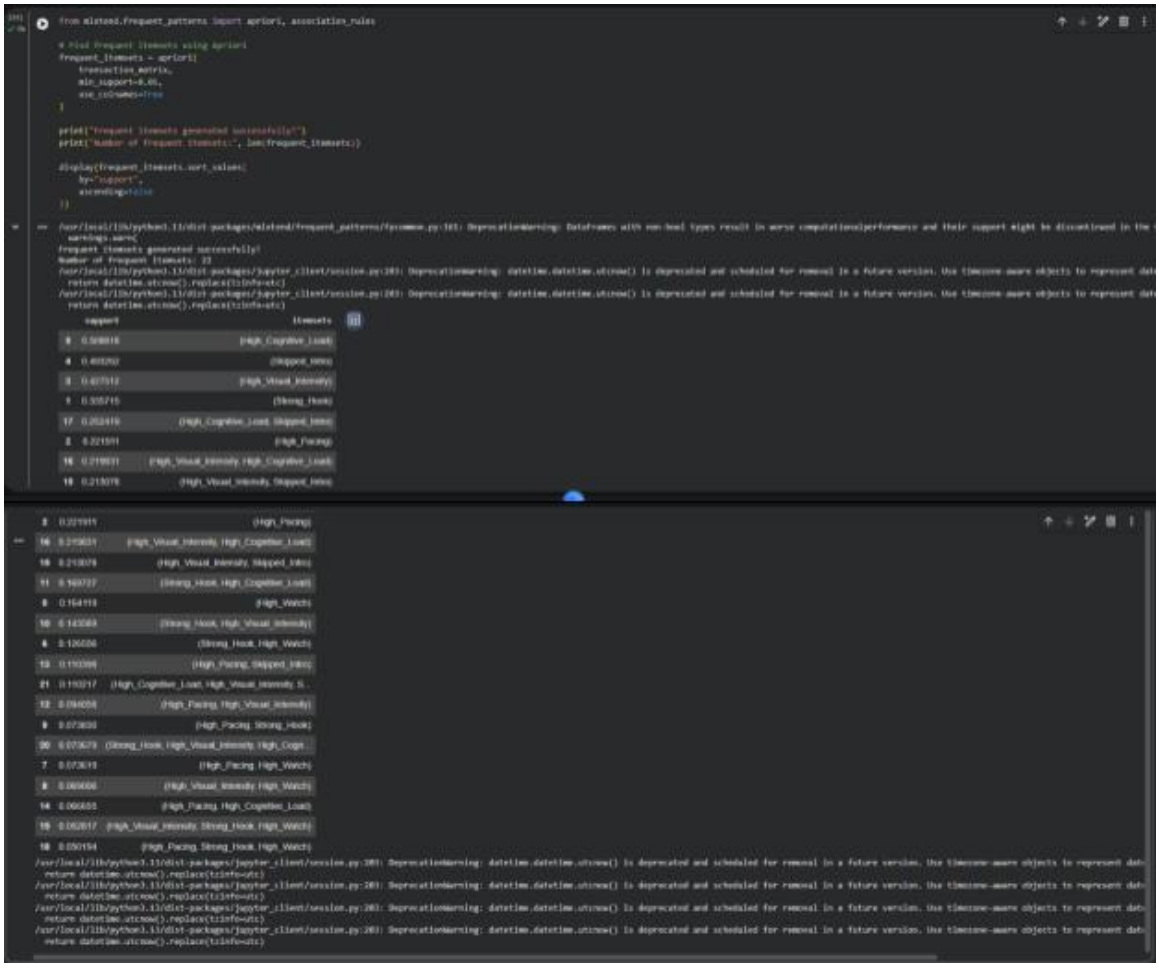


Figure 7: Frequent itemsets generated by Apriori (min_support = 0.05)

The Apriori algorithm was applied to identify frequent combinations of behavioural items. The resulting itemsets represent individual items or groups of items that occur frequently in the dataset. The displayed output provides additional frequent itemsets and their support values. These values help determine how commonly different combinations of viewer behaviours occur together.

```
# Generate association rules

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.30
)
```

```
# Keep only positive associations
rules = rules[rules["lift"] >= 1]

# Sort by confidence and then lift
rules = rules.sort_values(
    by=["confidence", "lift"],
    ascending=False
)

print("Association rules generated successfully!")
print("Number of rules:", len(rules))

display(
    rules[["antecedents", "consequents", "support", "confidence", "lift"]].head(10)
)
```

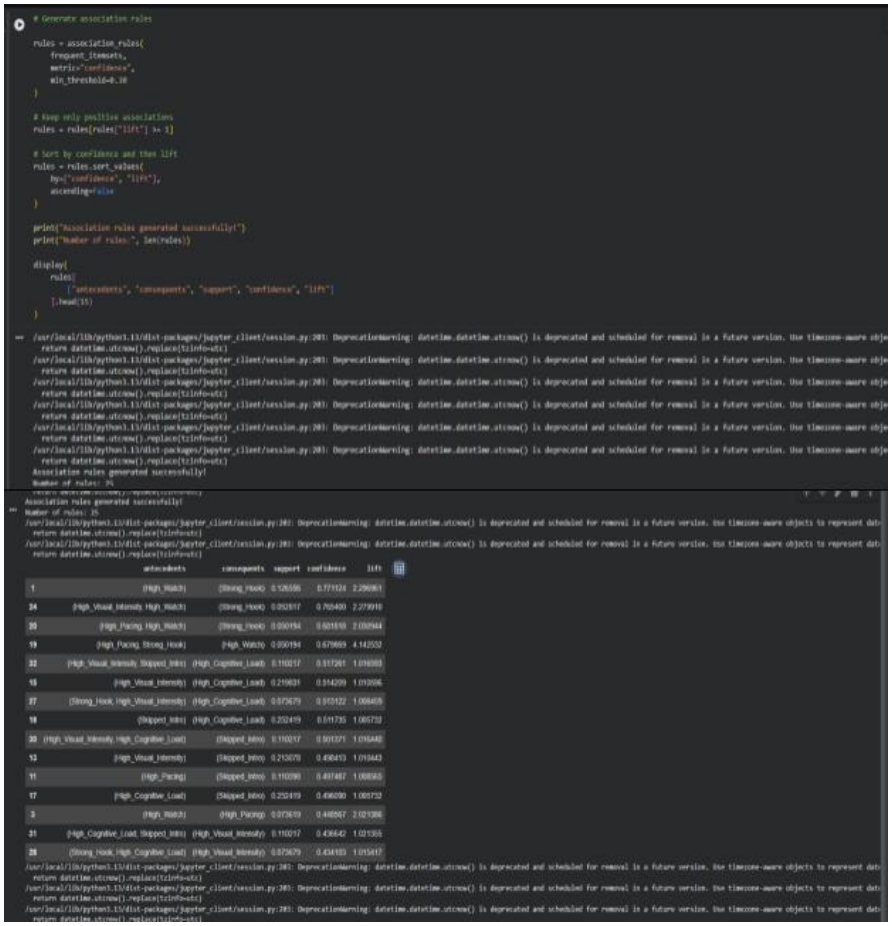


Figure 8: Association rules (confidence ≥ 0.30, lift ≥ 1)

Association rules were generated from the frequent itemsets. Each rule contains an antecedent (the starting condition) and a consequent (the associated item). These rules help identify relationships between viewing behaviours. The additional association rules were examined to compare different relationships among behavioural items. Measures such as support, confidence, and lift can help assess the frequency and usefulness of each rule.

```
# Format association rules for easy interpretation

top_rules = rules.copy()
```

```
top_rules["Antecedent"] = top_rules["antecedents"].apply(
    lambda x: ", ".join(sorted(x))
)

top_rules["Consequent"] = top_rules["consequents"].apply(
    lambda x: ", ".join(sorted(x))
)

final_rules = top_rules[
    ["Antecedent", "Consequent", "support", "confidence", "lift"]
].head(10)

final_rules = final_rules.round({
    "support": 4,
    "confidence": 4,
    "lift": 4
})

print("Top 10 Association Rules:")
display(final_rules)
```

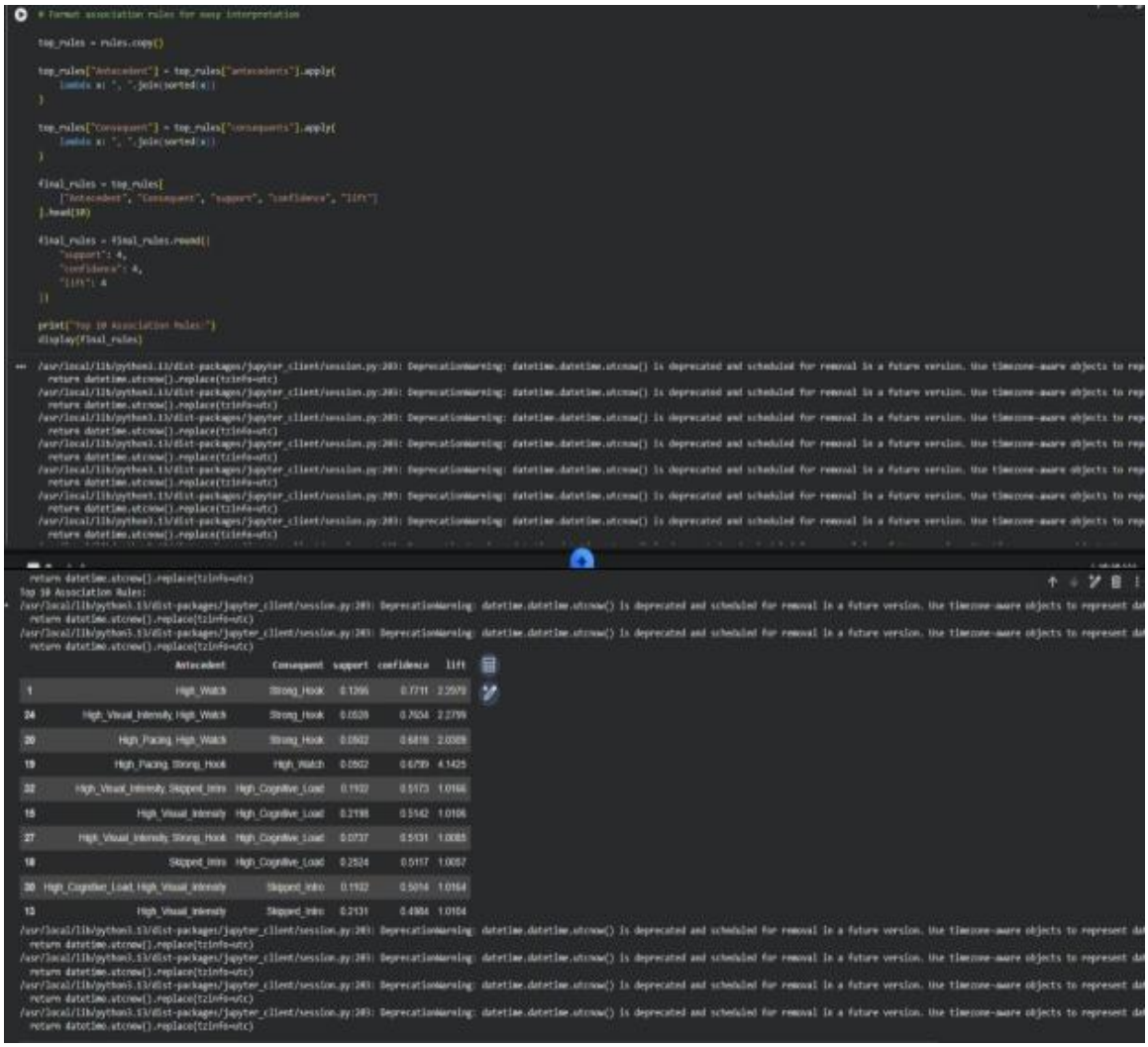


Figure 9: Top 10 association rules with support, confidence and lift

The association rules were ranked or filtered to identify the most relevant patterns. This reduces the number of rules that need to be examined and helps focus on relationships that may provide useful insights into viewer engagement. The remaining top rules provide further examples of relationships between behavioural characteristics. Comparing their support, confidence, and lift helps evaluate how frequently and strongly the items are associated.

```
import matplotlib.pyplot as plt

# Create rule labels
final_rules["Rule"] = (
    final_rules["Antecedent"] + " → " + final_rules["Consequent"]
)

# Plot confidence of top 10 rules
plt.figure(figsize=(10, 6))

plt.barh(
    final_rules["Rule"],
    final_rules["confidence"]
)

plt.xlabel("Confidence")
plt.ylabel("Association Rule")
plt.title("Top 10 Association Rules by Confidence")
plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```



Figure 10: Top 10 association rules by confidence (bar chart)

The association rules were represented graphically to make the relationships easier to interpret. Visualization helps compare rule strength and identify potentially interesting patterns in viewer behaviour. The additional visualization provides another view of the association rules. It supports comparison of different item combinations and helps interpret the discovered patterns more clearly.

```
# Find the strongest association rule based on lift

strongest_rule = rules.loc[rules["lift"].idxmax()]

print("Strongest Association Rule:")
print("Antecedent :", strongest_rule["antecedents"])
print("Consequent :", strongest_rule["consequents"])
print("Support    :", round(strongest_rule["support"], 4))
print("Confidence :", round(strongest_rule["confidence"], 4))
print("Lift       :", round(strongest_rule["lift"], 4))
```



Figure 11: Strongest association rule by lift

The strongest or most relevant association rule was identified from the generated results using the selected evaluation measure. The rule describes a relationship between behavioural items; however, association does not prove that one behaviour causes another.

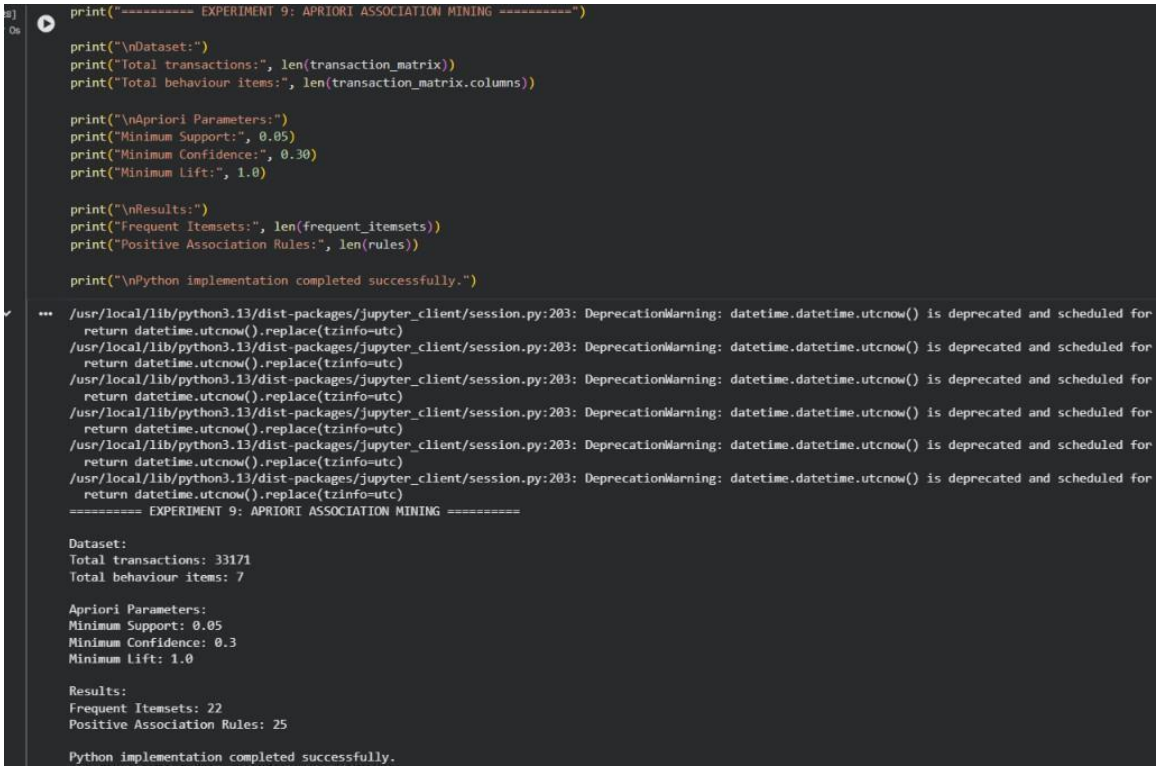
```
print("========== EXPERIMENT 8: APRIORI ASSOCIATION MINING ==========")

print("\nDataset:")
print("Total transactions:", len(transaction_matrix))
print("Total behaviour items:", len(transaction_matrix.columns))

print("\nApriori Parameters:")
print("Minimum Support:", 0.05)
print("Minimum Confidence:", 0.30)
print("Minimum Lift:", 1.0)

print("\nResults:")
print("Frequent Itemsets:", len(frequent_itemsets))
print("Positive Association Rules:", len(rules))

print("\nPython implementation completed successfully.")
```



```
print("========== EXPERIMENT 9: APRIORI ASSOCIATION MINING ==========")

print("\nDataset:")
print("Total transactions:", len(transaction_matrix))
print("Total behaviour items:", len(transaction_matrix.columns))

print("\nApriori Parameters:")
print("Minimum Support:", 0.05)
print("Minimum Confidence:", 0.30)
print("Minimum Lift:", 1.0)

print("\nResults:")
print("Frequent Itemsets:", len(frequent_itemsets))
print("Positive Association Rules:", len(rules))

print("\nPython implementation completed successfully.")
```

```
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
  return datetime.utcnow().replace(tzinfo=utc)
========== EXPERIMENT 9: APRIORI ASSOCIATION MINING ==========

Dataset:
Total transactions: 33171
Total behaviour items: 7

Apriori Parameters:
Minimum Support: 0.05
Minimum Confidence: 0.3
Minimum Lift: 1.0

Results:
Frequent Itemsets: 22
Positive Association Rules: 25

Python implementation completed successfully.
```

Figure 12: Summary of the Python Apriori run

The final Python output summarizes the results of association rule mining, including the frequent itemsets and the generated rules. These results demonstrate how data mining can reveal recurring combinations of viewing behaviours in the JioHotstar dataset.

Result

Table 1: Item frequency of the seven behaviour items (33,171 transactions)

Item	Count	Support %
High_Cognitive_Load	16,878	50.88
Skipped_Intro	16,362	49.33
High_Visual_Intensity	14,181	42.75
Strong_Hook	11,136	33.57
High_Pacing	7,361	22.19
High_Watch	5,444	16.41
Night_Watch_Safe	527	1.59

Table 2: Apriori parameters and output

Parameter / output	Value
Minimum support	0.05
Minimum confidence	0.30
Minimum lift	1.0

Frequent itemsets	22
Positive association rules	25
Strongest rule (by lift)	High_Watch → {High_Pacing, Strong_Hook}: support 0.0502, confidence 0.3058, lift 4.1425

The final output summarizes the experiment and its findings. Association rule mining helps identify frequent behavioural patterns and relationships that can support the analysis of viewer engagement and episode characteristics.

Conclusion

In this experiment, I learned how to use the Apriori algorithm for association rule mining. I understood how to identify frequent itemsets and generate association rules using support, confidence, and lift. This experiment helped me understand the relationships between different items in a dataset and how these patterns can be useful for decision-making. Initially, understanding the algorithm and interpreting the rules was a little difficult, but performing the experiment helped me understand the process better.

GitHub Folder: https://github.com/Sanjog-Poojary/DMBI-Lab/tree/main/Exp8_Apriori_Association